

HODLcoin

or, a final resolution to the infinite argument over whether saving in bitcoin is a form of using bitcoin.

(waxwing)

May 17, 2026

1 Introduction

Somsen's earlier work on spacechains had two essential components: burning coins on L1 (Bitcoin) and blind merged mining for consensus of L2, with the concept that the coins on L2, while not extractable back to L1, may hold value and may be swappable back to L1, contingent on the L2 surviving.

I considered that this framing was a bit "off" in a couple of ways. First, I think it's important to cleanly disassociate the issuance question from the consensus question. There are a ton of ideas about how to make L2 consensus work, from plain vanilla proof of work or proof of stake independent of L1, through SNARK based rollups and bridges, all the way to (blind) merged mining and drivechains; i.e. it's open to what extent you want to leverage existing consensus or even become part of it. The main part of the idea here, sets that question aside; but we will address consensus at the end, offering plausible solutions for this specific new model.

Mainly here, we are going to address the question of: can the minting of coins on L2:

- be anchored: based on existing Bitcoin
- be fair: not give privileged minting rights to any set of entities
- be scarce: have supply be limited; at least asymptotically incrementally zero, if not perfectly finite
- be permissionless: privileged entities should not control in any way the action of minting: it should be the sovereign bitcoin user themselves who performs the minting action, to prevent censorship or privacy loss

To abuse a horrible Claude-ism: "the word "based" is doing a lot of the heavy lifting". A classic holy-grail style L2 literally takes a Bitcoin utxo and interprets it as an L2 coin and then has some magical construction to ensure that the Bitcoin utxo can then only be spent in case of a succinct proof that the claimer is entitled to it, by L2 rules.

The design here is radically different: we intend to derive the value of the L2 coin from the **optionality** of spending the existing Bitcoin utxo.

As for scarcity and controlled supply, we are somewhat more relaxed, but derive an approximate control through something analogous to Bitcoin's difficulty retarget ratchet, as we shall see. The effect is that supply is strongly rate limited by how many Bitcoin exist, but that the

coin created is not literally a “parallel” bitcoin, but rather a derivative of existing Bitcoin’s value.

The core idea, in a nutshell: timelocked coins represent given-up optionality (the option to sell or transfer). This given up value is independently created and provable by any user, and so can be used as permission to mint new coins on an L2, henceforth “hodlcoin” for reasons that will become increasingly obvious over the course of this document.

Some immediate observations are worth making: 1/ supply is limited by the fact that Bitcoin utxos are finite in value. 2/ spacechains are essentially an asymptote of this idea: if you lock up your utxo for an infinite time, you should get the largest possible of that coin on the L2, since you have burnt them. 3/ no soft forks or other intrusions on L1 are necessary for this issuance scheme; the consensus mechanism of L2 is another question that we’ll address later 4/ there is a happy concordance with the “hodling” philosophy of many bitcoiners: any user other than you who chooses to use this method is reducing the supply of bitcoin available for sale, thus theoretically increasing the value of your “hodlings”, and a more subtle but real effect: if most of your “usage” of bitcoin is in the form of saving it (cue: endless arguments), then this system is also useful to you for the minor amount of spending you do: because if you lock up a large stash, you, for free, get an additional small stash of L2 coins to do your paltry amount of spending! How large/small is a matter of mathematics that we’ll get into shortly.

One more issue in this early outline to mention: privacy and independence. The fourth bulletpoint raises something I think is crucial: it’s very useful to be able to interact with such a system without someone else’s intervention: for example, a centralized sequencer, or perhaps a swap provider and a market to trade. It’s not just that a swap provider takes a cut — which is often large, by the way! — or that they might be down or the market might not be liquid, it’s also something really important: most users want maximal privacy in their transactions. Achieving this is very non-trivial with Bitcoin, but in this system there is a path to making reasonably strong L1-L2 privacy: by choosing fixed timelocking periods, you can join an anonymity set such that your minting might be indistinguishable from the minting of a large anonymity set of other users. This is only a partially worked out idea, I won’t completely develop it in this doc.

1.1 Technical outline

Here’s a very brief summary of the (very simple) technical idea; you can skip this if you’re not a Bitcoin engineer and go to the next section on comparisons, because you’re probably wondering “how is this different from X?”.

An L2 mints new native tokens to users who provably commit a Bitcoin taproot UTXO for a chosen duration via `OP_CHECKSEQUENCEVERIFY` (CSV), with a per-mint reward $f_{\text{mint}}(V, T)$ that is superlinear in lock duration T for moderate T and saturates at the locked value V as $T \rightarrow \infty$. The exact formula will be outlined and justified later, but for now note that the formula will have only one free variable r representing an interest rate.

The taproot scriptPubKey contains an additional leaf with an unspendable Tagged Hash of the pubkey, where the tag is a label that specifies the L2. That means that this utxo is only authorized for this L2, such as “hodlcoin”.

The vanilla minting will just expose the pubkey and the timelock along with the outpoint, so that L2 nodes can reconstruct the scriptPubKey and verify valid minting. A ZKP will have to be developed, along with nullifiers to make this minting event k-anonymous for some fixed epoch like 3 months.

The final technical point is control of the issuance schedule. As mentioned, there is a parameter r , roughly an interest rate, that controls what amount of L2 is minted per L1 (time, value) lock. If issuance increases (it cannot be controlled, remember, since there is no entity with that power), r decreases to make issuance less economically attractive; and vice versa. This, like bitcoin's difficulty retarget, allows a reasonably controlled supply, though not a fully controlled one. A reasonable target is a constant issuance per epoch (which asymptotically will reduce new issuance to near zero).

1.2 Comparisons

- Altcoins: Later in the doc, we'll talk about ex-nihilo money creation. Bitcoin did this, but fairly: a lottery based on proof of work. Later altcoins did this less and less frequently and purely, eventually most of them used an "issue to insiders and then sell" model, mostly. But even the fairest are indeed creating money from nothing unless they are using proof of work. Energy supply is not infinite, therefore the security budget of coins is not infinite. This hodlcoin model does not create money from nothing: it takes the optionality of bitcoin holding and replaces it with a token: there is, in an abstract sense, a trade going on: by timelocking your coins, you are giving value to all other Bitcoin holders, who gain from having less selling competition (both if they want to sell, and if they don't, there is a gain to them here; bitcoins are literally more scarce during the timelock). Your trade is: I get an L2 coin I can use for <insert use case> while I lose something I handed over to other Bitcoin holders: the right to sell for some other good.
- Spacechains: this is the easy one. As noted at the start, this idea is a development of that idea: spacechains are just the asymptotic case of hodlcoins: as $T \rightarrow \infty$, the value sacrificed is the entire coin. Indeed, one could easily imagine this as a preferred option in certain cases, **only if** the L2 has gained significant popularity/usage. The nice thing about hodlcoin is it's so low risk: you could get some usage while spending literally no bitcoin, only spending some very minor optionality that you quite likely wouldn't have used anyway. The consensus proposal (blind merged mining) in spacechains, we'll discuss nearer the end.
- Utxo snapshots: this comparison is instructive. You can create a new token/L2 by saying "every holder of bitcoin at date T gets the proportionate amount on L2". This has the fairness property but not the supply limit/scarcity property: since no coins are locked, there is no sacrificed value. Even one such "snapshot-coin", let alone 1M of them, represents ex-nihilo money creation. To be clear this is not saying "you can't do that" — clearly, you can — but it's arguing against the market's placing value on it.
- Drivechains: it's addressing the consensus part, which I have already said here, is not being addressed by this design; this design is focusing on what might be workable for the **issuance** question. Not to say that there isn't overlap; I'll later propose a more simplistic consensus approach that doesn't require a soft fork. The part of drivechains that requires the soft fork is related to the problem of L2 to L1 exit, which hodlcoin very explicitly sidesteps.
- Validity rollups: again, it's not a direct comparison, but there are important overlaps to consider: rollups were more conceived for chains like Ethereum, in which state transitions are more easily enforced directly with L1 validation. In Bitcoin, that isn't possible, except with BitVM et al as per the next item, but curiously this doesn't matter as much as you'd think: if a state transition is succinctly verifiable by even light clients or full nodes on L2, then a cheating sequencer does not profit from publishing

garbage state update attestations on L1, while the main point of attestation: borrowing from Bitcoin’s double spend prevention, protected by proof of work, is still there. We’ll discuss this more later in the doc, but for now, note that the other part of the concept of a rollup: using a DA layer to publish concise transaction representations, is somewhat orthogonal: it can be done in this hodlcoin construction, with a state root committed to L1 trivially in OP_RETURN or similar.

- **BitVM and friends:** BitVM and Glock are hyper-focused on the no-L1-changes solution to the near-insoluble problem of “verify L2 state on L1”. As such they (attempt to) solve an extremely important problem as trustlessly as possible; even where they are practical (arguably Glock/Mosaic are starting to approach that), they have to trade off on trust a little bit: signing committees in the absence of covenants in Bitcoin Script, and probably more importantly, named Operator entities to execute withdrawals - this element is probably the biggest weakness of the idea, since exit implies forming relationships with Operators (or being one!).

It might be worth mentioning the trivially true: any such L2 can be exited out of via atomic swap actions, assuming you can find a counterparty. (As was observed in spacechains, and later in BitVM also). This could be seen as the “obvious” way to participate if you only care about the economics, not the payment functionality: mint the L2 coins by locking, then immediately swap out of L2 to L1 for a (probably very small) extra L1 profit. This is not a flaw: it’s a natural market mechanism for people to express their preference for L1 holding over whatever functionality L2 offers (such as private or cheap transactions or etc.).

What’s the biggest trade-off that the hodlcoin idea makes? Simple, it’s that you only gain access to a small fraction of your bitcoin’s value, unless you trust the L2 so much that you want to move all your capital across permanently. That’s why “hodlcoin” an otherwise silly name, fits particularly well. That slider of ‘a small fraction’ is to my mind an enormous advance on the earlier Spacechains idea.

2 The Mechanism

Let u denote a Bitcoin taproot [2] UTXO of value V , constructed as follows by the locker. The locker:

1. Selects an internal key Q that is a Nothing-Up-My-Sleeve (NUMS) point, i.e., a curve point whose discrete logarithm is provably unknown. We recommend the standard NUMS point specified in BIP341. The choice of a NUMS internal key ensures that u is not spendable via the keypath; the only way to spend u is by revealing a script-path leaf, which forces the locker to honour the CSV constraint defined below.
2. Constructs a tapscript Merkle tree containing (at least) the following two leaves:

Spending leaf L_{spend} :

$\langle T \rangle$ OP_CHECKSEQUENCEVERIFY OP_DROP $\langle pk \rangle$ OP_CHECKSIG

where T is the chosen relative locktime (the committed duration, in blocks) and pk is the locker’s BIP340 x-only Schnorr public key. This leaf is spendable T blocks after u is confirmed, by the holder of the corresponding secret key.

Why CSV rather than CLTV. We use OP_CHECKSEQUENCEVERIFY (CSV, BIP112) rather than OP_CHECKLOCKTIMEVERIFY (CLTV, BIP65) because the minting function takes

the relative duration T as its argument, and CSV encodes that quantity directly in the script. A CLTV-based construction would commit instead to an absolute unlock height T_{unlock} , requiring the L2 to derive $T = T_{\text{unlock}} - T_{\text{create}}$ via a chain-state lookup, and would expose the locker’s intended duration to mempool-confirmation latency (since T_{create} is only known after confirmation). CSV avoids both: the locker chooses a duration, the script records that duration, and the L2 reads it directly.

Data (namespace) leaf L_{data} :

`OP_RETURN  $\langle D \rangle$`

where $D = \text{TaggedHash}(\text{"L2/chain_id/v1"}, pk)$ is a 32-byte commitment binding u to a specific L2’s namespace. Tapscript inherits Bitcoin’s rule that `OP_RETURN` aborts script execution, so L_{data} is permanently unspendable: it exists in the tree solely as a committed data payload, never executed. The rationale for its placement (in a tapscript leaf rather than in an on-chain `OP_RETURN` output at funding time) is privacy, and is discussed in Section 6.

3. Computes the Merkle root R of the script tree and the tweaked output key $Q' = Q + \text{TaggedHash}(\text{"TapTweak"}, Q \parallel R) \cdot G$. The locker then funds the P2TR output `OP_1  $\langle Q'_x \rangle$` with V satoshis.

Let T_{create} denote the block height at which u was confirmed. The committed duration is T , as fixed in L_{spend} , and the active lock period is the half-open interval $[T_{\text{create}}, T_{\text{create}} + T)$ during which u is unspendable.

Minting on L2. A user who controls pk may, at any block height h during the active lock period, submit to the L2 a minting request comprising:

- the outpoint of u , together with the claimed value V ;
- the internal key Q (which the verifier checks is the NUMS point) and the Merkle root R ;
- the scripts of L_{spend} and L_{data} , each with a Merkle path to R ;
- a Schnorr signature under pk on a minting message

$$m = (\text{outpoint}(u), h, L2\text{-destination}).$$

The L2 verifies in sequence:

- (i) $\text{outpoint}(u)$ exists in the L1 UTXO set with value V and output script `OP_1  $\langle Q'_x \rangle$` , where Q' is recomputed from Q and R as above;
- (ii) Q is the canonical NUMS point;
- (iii) the Merkle path for L_{spend} resolves to R , L_{spend} parses as the script above with a positive integer T , and the embedded pk matches the signing key;
- (iv) the Merkle path for L_{data} resolves to R , L_{data} parses as `OP_RETURN  $\langle D \rangle$` , and $D = \text{TaggedHash}(\text{"L2/chain_id/v1"}, pk)$ for the L2’s own `chain\_id`;
- (v) the Schnorr signature on m verifies under pk ;
- (vi) $\text{outpoint}(u)$ has not previously been consumed by a mint (the L2 maintains a set of consumed outpoints).

On successful verification, the L2 credits the L2 destination with

$$f_{\text{mint}}(V, T) = V(1 - (1 + rT)e^{-rT}) \quad (1)$$

units of the L2 native token. The single parameter r is set in L2 consensus and retargeted as described in Section 5.

Unlock. At block $T_{\text{create}} + T$ or any later height, the locker recovers the funds by performing a standard script-path spend of u via L_{spend} , signing with the secret key for pk and setting **nSequence** appropriately for CSV to permit the spend. This is a normal Bitcoin transaction; nothing about it is L2-specific. The L2 protocol takes no action.

Several properties of this construction deserve emphasis up front:

- The Bitcoin UTXO is never spent by the L2 protocol. After $T_{\text{create}} + T$ it is unconditionally spendable by its owner under Bitcoin’s existing rules. The L2 makes no claim on it.
- During the lock period, the on-chain footprint of u is exactly one P2TR output — an output indistinguishable from any other taproot output, regardless of what its hidden script tree contains. The existence of the lock is not advertised on L1.
- The L2 token minted is not BTC, and is not pegged to BTC. It is a separate token whose issuance is metered against demonstrated, time-bounded BTC commitment. Its value is doubtless strongly correlated with BTC value, but it’s not the same thing.
- There is no committee, federation, or threshold quorum involved in the issuance mechanism.

3 Philosophical Implications of Issuance

In the introduction, we mentioned the question of ex-nihilo money creation in different models, and argued that it doesn’t apply here. But that’s a nontrivial point so I think it’s worth exploring in detail. Here goes.

The construction described above appears, at first reading, to create new value out of nothing. The locker retains their BTC; they additionally receive L2 tokens; nothing was destroyed. Naively, this looks like ex-nihilo money creation. This section argues that the construction is better understood as the formal recognition of a value transfer that already occurs as an unattributed externality in the underlying market, and that the L2 token is the compensation due to the party who generates that externality. We make this argument in stages, then connect it to the spacechain proposal, which is the limiting case of the present construction.

3.1 What was actually exchanged

The locker did not retain the same asset they began with. They began with V of liquid BTC and ended with V of locked BTC together with some quantity of L2 tokens. Liquid BTC and locked BTC are not the same asset: the liquidity itself, that is, the option to deploy V at any moment during the interval $[0, T]$, has economic value, recognised in any options-pricing framework. The locker has therefore shorted this optionality and received L2 tokens in compensation. The principal V is preserved; the optionality on V over $[0, T]$ is not.

What is unusual is the apparent absence of a counterparty. In an ordinary trade, the asset received by one party was held in advance by the other. The L2 tokens, by contrast, did not exist before the minting event; they were brought into existence by the act of locking. There is no party who possessed the L2 tokens beforehand and yielded them in exchange. Naively,

then, the construction looks like an asymmetric trade: the locker has paid (in foregone optionality) and the protocol has issued, but nobody has received the payment.

3.2 The unattributed counterparty

The counterparty exists, but it is diffuse and unsigned. When a holder withdraws supply from circulation by locking it, the remaining holders of the same asset benefit from the reduced free float. The benefit is real and continuous: it manifests as reduced steady-state selling pressure, reduced expected slippage on any future sale, and, reflexively, an improved scarcity narrative for the asset as a whole. These benefits accrue to every holder of unlocked BTC during the interval $[0, T]$, in proportion to their holdings.

The benefit is, in the language of welfare economics, a positive externality. The locker bears the full cost of their lock (foregone optionality), while the benefit (reduced float) is diffused across the unlocked-holder population without compensation. Under ordinary market dynamics, this externality is unattributed and uncompensated. No one writes a cheque to the locker for the slight price-support they conferred on the rest of the market.

The L2 issuance scheme can be read as the formal internalisation of this externality. The protocol does not acquire any asset against the tokens it issues, but the tokens are not, in any meaningful sense, conjured from nothing. They are the receipt for a transfer of value that already occurs in the underlying market, made attributable by the protocol's verification of the locker's act. The L2 protocol does not create the externality; it merely makes the externality compensable.

3.3 The free-rider correction

This framing also explains, at the level of incentives, why such a mechanism is necessary at all. The supply-reduction externality is a public good: each unlocked holder benefits regardless of whether they themselves contributed to producing it. Standard free-rider dynamics imply that the externality will be underproduced relative to the social optimum, because each potential locker captures only a small fraction of the benefit they generate while bearing the full cost.

The L2 token is, in this reading, a coordination mechanism that overcomes the free-rider problem. By making issuance contingent on locking, the protocol offers a private benefit (the L2 token) that internalises some fraction of the public benefit (price support for remaining holders) that the locker generates. Rational lockers will participate up to the margin at which the L2 tokens received, plus their share of the price-support benefit on their own retained holdings, equals the optionality cost of the lock. The total amount of locking that emerges is therefore endogenous and bounded by the market's revealed valuation of supply reduction: if holders do not in aggregate value reduced float, no one locks; if they do, locking emerges in proportion.

Attribution requires binding. The externality-compensation interpretation depends on each lock being uniquely attributable to a single L2's compensation regime. If a locker could simultaneously claim against several L2s with the same underlying lock, the marginal locker's participation would no longer reveal a per-L2 valuation: it would reveal a sum, with no per-L2 decomposition. The price-discovery signal that gives the L2 market its information would degrade. The cross-L2 binding mechanism developed in Section 6 — a namespace commitment carried by the funding output, restricting each lock to one L2's issuance — is therefore not merely an anti-cheating measure but a precondition for the present interpretation to hold.

A locker who binds to chain A is making a credible declaration that this particular act of locking is uniquely attributable to chain A , and that the optionality they have shorted is being offered, in full, in exchange for chain A 's tokens. The externality itself accrues to all BTC holders regardless of which L2 the locker chose; the binding selects only which compensation channel is opened in exchange.

3.4 Spacechain as the asymptotic case

The spacechain proposal [1] is the $T \rightarrow \infty$ limit of the present construction. A burn permanently removes V from BTC circulation; the externality conferred on remaining holders is correspondingly permanent. The locker in the spacechain construction has shorted not the optionality on V over a finite interval but the entirety of V itself, and the externality they generate is the unending price-support effect of permanently destroyed supply.

Viewed through the lens of this section, spacechain and the present construction are the same mechanism evaluated at different points: both compensate a locker for a supply-reduction externality, and the magnitude of compensation is proportional to the magnitude of the externality generated. Where spacechain offers users a binary choice between zero externality and total externality, the present construction parameterises the choice continuously, with $f_{\text{mint}}(V, T)$ tracking the magnitude of the temporary externality that locking V for duration T produces. The saturation of f_{mint} at V as $T \rightarrow \infty$ is therefore not merely a mathematical convenience: it expresses the fact that a temporary lock asymptotes to a permanent burn, and a permanent burn is the maximal externality the locker can produce.

This places both proposals in a single lineage of mechanisms that compensate generators of positive externalities: payments for ecosystem services, carbon-sequestration credits, and similar Coasean instruments. The construction differs from these only in that the externality being compensated is the supply-reduction effect of voluntary lockup on an asset's market float.

3.5 What the protocol does and does not claim

We are careful to note what this framing does, and does not, justify.

It justifies the issuance, not the value. The L2 token is compensation for a real act with real economic consequences. This justifies the protocol's choice to issue tokens conditional on locking; it does not, by itself, give the L2 tokens any particular market value. Whether the tokens come to be worth holding depends on whether the L2 develops utility, applications, demand, or speculative interest after the fact. The issuance is honest and grounded; the resulting tokens may or may not flourish.

It does not require an L1-side opinion. The Bitcoin protocol need not, and does not, acknowledge any of this. The externality exists at the market level, not the protocol level. Bitcoin sees only that some UTXOs are time-locked. The economic interpretation of that locking, and the L2 issuance that the present construction triggers in response, take place entirely on the L2 side. No claim on Bitcoin's accounting or semantics is required.

It does not require the externality to be large. A common objection to such framings is that the supply-reduction effect of a single lock is too small to constitute meaningful value transfer. This is true at the level of individual locks but irrelevant in aggregate: the externality summed across all locks during $[0, T]$ may be substantial, and the question of marginal valuation is settled by lockers themselves through their participation decisions. If

lockers do not believe the L2 tokens are worth the optionality foregone, they do not lock, and the mechanism produces no issuance. The market reveals whether the externality, on its current margin, is worth compensating.

The construction is not ex-nihilo in the loose sense. Where “ex-nihilo” is taken to mean “free money produced without any underlying real cost or counterparty,” the construction does not qualify. There is a real cost (forgone optionality) borne by a real party (the locker), in exchange for a real benefit (reduced float) conferred on a real population (unlocked holders), with the L2 token serving as the formal record of the exchange. The construction is ex-nihilo only in the narrow protocol-accounting sense in which every fair-launched cryptocurrency, including Bitcoin’s own block-subsidy issuance, is ex-nihilo: the protocol does not acquire an asset against the tokens it issues. This narrow sense applies equally to mining, snapshot airdrops, and burns; it does not distinguish the present construction from any other fair-launch scheme, and it does not carry the pejorative connotations sometimes attached to “money created from nothing.”

4 The Minting Function

The functional form of f_{mint} is the most opinionated choice in this design. We motivate it by identifying two failure modes at opposite ends of the lock-duration range, and showing that they jointly force a sigmoidal shape.

4.1 Two failure modes

Anti-griefing (short T). If short locks were rewarded proportionally to T , or worse, super-proportionally per unit time, an adversary could grief L1 by manufacturing many short-duration UTXOs — individually negligible in commitment, collectively flooding both L1 UTXO state and L2 issuance. The minting function must therefore be superlinear in T for short and moderate values of T : doubling T must more than double the reward, so that consolidating commitment into fewer, longer locks strictly dominates fragmenting it. The slope at the origin should additionally be zero, so that arbitrarily short locks earn arbitrarily little.

Anti-monopolisation (long T). The limiting case $T \rightarrow \infty$ is economically equivalent to burning the UTXO: the owner has unconditionally given up access to V . The maximum sacrifice the protocol can verify is V . If f_{mint} exceeded V in the long- T limit, then a sufficiently determined actor could capture an arbitrarily large fraction of L2 supply by committing a small amount of BTC for long enough — the protocol would be assigning unbounded L2 value to a bounded L1 sacrifice. The minting function must therefore be bounded above by V , with V achieved in the burn-equivalent limit.

4.2 Why both requirements are necessary

The two requirements are jointly satisfied only by a function that is superlinear in the short-to-medium range and saturating at the tail — a sigmoid in T . No smooth function can be both globally superlinear and bounded: globally superlinear functions diverge by iteration, $f(2^n T) > 2^n f(T) \rightarrow \infty$. We must therefore concede superlinearity somewhere, and the natural place to concede it is in the saturation regime, where the marginal reward is already small by construction.

4.3 Why bounded by V rather than by an opportunity-cost integral

A reader from financial economics will object that the “true” value of a long commitment is the foregone yield, $V(e^{rT} - 1)$ for some assumed rate r ; this is unbounded and grows exponentially with T . We reject this functional form for three converging reasons.

Epistemic. The protocol can verify, on-chain, that a UTXO is locked. It cannot observe the rate at which BTC could be productively deployed elsewhere; that rate is unobservable, contested, and varies between actors. Any value the protocol picks for such a rate is a fiction it imposes on the locker. The protocol should reward what it can verify, and what it can verify is the fraction of principal irrevocably committed — not the counterfactual yield.

Structural. The protocol is not a counterparty to the locker. It owes the locker nothing; it does not promise compensation for opportunity cost. What it is doing is rationing scarce L2 issuance among competing claimants, and it requires a yardstick that ranks claims consistently. “Fraction of V irrevocably committed” is such a yardstick: bounded, comparable across lockers, and grounded in observable on-chain state. “Foregone yield at some imagined rate” is not.

Self-consistency. If the reward exceeded V in the long- T limit, the protocol would be claiming that committing V for long enough is worth more L2 issuance than burning V outright. But burning is the strictly more sacrificial action — the owner has eliminated access rather than deferred it. A reward function that pays more for the less-sacrificial action is not monotone in actual sacrifice. The bound at V is what keeps the reward function monotone, with burning as the limiting case.

4.4 The chosen form

The simplest closed-form sigmoid satisfying these requirements is

$$f_{\text{mint}}(V, T) = V(1 - (1 + rT)e^{-rT}).$$

We note its properties:

- $f_{\text{mint}}(V, 0) = 0$ and $f'_{\text{mint}}(V, 0) = 0$. Trivial locks earn essentially nothing; the function ramps quadratically from the origin, as $V(rT)^2/2 + O(T^3)$.
- $f_{\text{mint}}(V, T) \rightarrow V$ as $T \rightarrow \infty$. Burning is the limiting case, and the spacechain construction is recovered exactly in this corner.
- The function has a single inflection point at $T = 1/r$. Below the inflection it is convex in T (super-additive: anti-splitting); above it, concave (sub-additive). For realistic parameter choices the convex regime spans years to decades, comfortably containing the durations users will choose in practice.
- Only one parameter, r , which is set in consensus.

We do not claim that this is the unique sigmoid satisfying the design requirements. Other choices — $V \tanh^2(rT)$, $V(rT)^2/(1 + (rT)^2)$, the Erlang-3 CDF, etc. — have the same qualitative properties. We pick (1) for its compactness and for the direct relationship between its tail behaviour and the discount factor e^{-rT} familiar from present-value calculations: the surplus mass $f_{\text{mint}}(V, T) - V \cdot (1 - e^{-rT})$, which is exactly $V \cdot rT \cdot e^{-rT}$, makes precise the additional credit awarded for sustained (rather than rolling) commitment.

5 Retargeting r

The remaining question is how r is chosen. There is no objective answer: r sets the inflection point of f_{mint} , equivalently the lock duration at which marginal reward begins to taper. Any constant choice will be too generous in some market conditions and too stingy in others.

The natural solution is to ratchet r in the manner of Bitcoin’s difficulty retarget. The L2 commits in consensus to a target rate M^* of newly minted tokens per L1 block. Within each retarget window of W blocks, the L2 measures the actual mint volume M . If $M > M^*$, r is decreased so that future locks mint fewer tokens for the same (V, T) ; if $M < M^*$, r is increased. The adjustment per window should be capped (Bitcoin uses a $4\times$ clamp) to prevent oscillation.

Three observations deserve to be made explicit:

The target M^* is arbitrary. As with Bitcoin’s 10-minute block target or 21M cap, the absolute number is a unit choice; what matters is that it is fixed, credibly committed in consensus, and stable. It determines the long-run scarcity of the L2 token.

The retarget operates on a flow that is purely positive. New mints add to L2 supply; L1 unlocks do not destroy L2 tokens. Once minted, an L2 token is an ordinary L2 asset whose subsequent fate is unrelated to the underlying L1 UTXO. This makes the retarget structurally simpler than one would naively expect: the controlled variable (cumulative L2 supply) is monotone, exactly as in Bitcoin.

Adjustment lags response. Unlike Bitcoin miners, who can re-deploy hashpower within minutes, prospective lockers responding to r changes must wait for new time-locked UTXOs to confirm and for their chosen lock durations to elapse. Locks already in flight do not respond to r changes; only new locks do. The retarget therefore has a built-in lag proportional to the average lock duration. This argues for retarget windows of months rather than the weeks-scale used by Bitcoin’s difficulty adjustment.

6 Binding and Reuse Prevention

The construction in Section 2 is designed to resist two distinct forms of misuse, which are orthogonal and require separate mechanisms.

6.1 Cross-L2 binding via the data leaf

Absent any binding, the same time-locked UTXO u could be presented to several competing L2s as proof of eligibility, with each L2 issuing its own native tokens against the same underlying lock. The locker would bear the cost of the lock once (one act of foregone optionality) but receive compensation from several L2 economies in parallel. This is not merely an attack on any single L2; it is a corruption of the price-discovery signal that the construction relies on (Section 3): the marginal locker would no longer be revealing a per-L2 valuation but a pooled valuation across all L2s they had simultaneously claimed against. No L2’s market could read its own signal cleanly.

The cross-L2 binding mechanism resolves this by requiring each UTXO to commit, at funding time, to a specific L2 namespace. The data leaf L_{data} of Section 2 carries the commitment $D = \text{TaggedHash}(\text{“L2/chain.id/v1”, } pk)$, where the tag string includes the L2’s unique identifier. An L2 verifying a mint computes the same tag against its own `chain.id` and

rejects any mint whose L_{data} does not match. A UTXO stamped for chain A cannot mint on chain B , because B 's computation of D uses B 's `chain_id` and produces a different 32-byte digest.

Why a tapscript leaf rather than an on-chain OP_RETURN at funding time. An OP_RETURN output on the funding transaction would carry the namespace commitment in plain view on L1, visible to any node, indexer, or analytics firm. Every participating UTXO would announce itself as “a participant in the chain_id X issuance scheme” at funding time. Placing the commitment inside a tapscript leaf instead defers any disclosure: during the lock period, the on-chain footprint is a vanilla P2TR output, and the existence of L_{data} (and indeed of L_{spend}) is not advertised. The leaf becomes visible only when the locker reveals it during a script-path spend at unlock time, or when they submit a mint proof to the L2.

Why the stamp is keyed by pk rather than a protocol constant. A naive design would set $D = \text{TaggedHash}(\text{“L2/chain_id/v1”})$, a constant across all lockers of a given L2. The leaf hash $h(L_{\text{data}})$ would then be identical for every locker. An observer who acquired any single instance of this hash — for example by seeing it as a sibling in a Merkle path during another locker's script-path spend — could subsequently identify every other locker's leaf hash by direct comparison. The fixed protocol fingerprint would be trivially recognisable in any Merkle path containing it. Keying D to pk makes each locker's L_{data} unique, and its hash one of many indistinguishable random 32-byte values from an outside observer's perspective.

Cross-ecosystem standardisation. The mechanism's anti-double-claim guarantee is per-L2: chain B refuses chain A 's stamps if and only if B implements the check. A future L2 ecosystem participant who declines to check the stamp accepts UTXOs regardless of their namespace, including those already claimed by other L2s. Such an L2 externalises noise onto the wider ecosystem. The natural defence is a shared format: all participating L2s agree that the data leaf is to be parsed as OP_RETURN $\langle D \rangle$ with D derived from a tag string of the form “L2/chain_id/v1”, and each L2 verifies that the `chain_id` portion matches its own. Compliance is voluntary but legible.

6.2 Intra-L2 reuse prevention

The above prevents one UTXO from being claimed by multiple L2s. A separate concern is that the same UTXO might be claimed twice on the same L2 by submitting two minting messages. The basic, non-anonymous defence is straightforward: the L2 maintains a set of consumed outpoints, and any mint whose outpoint is already in the set is rejected. The minting message commits to the outpoint and is signed under pk , so the binding from outpoint to mint is unforgeable. Section 2 specifies this check as part of verification step (vi).

In a future anonymous variant, where the mint proof does not reveal the outpoint, the analogous defence is a nullifier: a key image in the LSAG style of Monero [6] or a nullifier in the style of Zcash. The aut-ct library [7] realises this construction over taproot. The L2 maintains the set of consumed nullifiers and rejects any mint whose nullifier is already present. The two reuse-prevention mechanisms (outpoint set, nullifier set) are functionally interchangeable; they differ only in what they reveal.

6.3 Anonymity properties: what the design delivers, and what it defers

We aim for a concrete first deployment with L1 anonymity that is strong in the common case but not exhaustive, and we ensure the construction is shaped to accept a full ZK upgrade later without redesign. We catalogue the leakage explicitly.

During the lock period: full L1 anonymity. On L1, u is a P2TR output. Nothing about its script tree is visible; no observer can distinguish u from any other taproot output. The set of time-locked taproot outputs across Bitcoin is large and grows for reasons unrelated to this construction (vaults, savings, inheritance, mining pool reserves, etc.); the L2's locks blend into that population.

At mint time: leakage to L2 viewers, not to L1. The mint proof is submitted to the L2 only. L1 sees nothing. L2 viewers (anyone reading the L2 commitment chain) see the outpoint, V , T , pk , and the L2 destination. The mapping from L1 UTXO to L2 destination is thus visible to L2 analytics in the basic version; L1 analytics is unaffected.

At unlock time: residual L1 identifiability. To recover V , the locker performs a script-path spend revealing L_{spend} and its Merkle path. The path necessarily includes (at minimum) the hash of L_{data} as a sibling. The script of L_{spend} exposes pk . An L1 observer can recompute $D' = \text{TaggedHash}(\text{"L2/chain_id/v1"}, pk)$ for any `chain\_id` of interest, hash this as a candidate L_{data} leaf, and compare to the revealed sibling hash. A match identifies the UTXO as having served the corresponding L2.

This is a real residual leakage. It does not affect any participant during the active lock; it affects only those who choose to recover their BTC via the script path after the lock expires (i.e., at or after block $T_{\text{create}} + T$), and only against observers who specifically probe for the L2's stamp format. We accept this in the first version. Two distinct future improvements close it:

- Decoy leaves and tree shaping. The tapscript tree may be extended with additional leaves so that the Merkle path from L_{spend} to the root does not necessarily include the hash of L_{data} . With careful tree layout, the L_{data} leaf can be placed in a separate subtree whose hash is summarised in the path but whose contents and direct leaf hash are not revealed by an L_{spend} spend. This reduces but does not eliminate the leakage.
- ZK proof at mint, alternate spend path at unlock. A ZK proof at mint time (Section 6.4) eliminates the L2-side mapping; combined with a script-tree design that admits an unlock spend not revealing L_{data} 's hash directly, this can drive residual L1 leakage to negligible levels. Full anonymisation is deferred.

6.4 The design is ZK-upgradeable

The construction above is deliberately structured so that the verification performed by the L2 can be re-expressed as a single statement amenable to a succinct zero-knowledge proof. The statement is:

"I know an outpoint o on L1 with output script $OP_1 \langle Q'_x \rangle$ and value V ; a NUMS internal key Q and a Merkle root R such that $Q' = Q + \text{TaggedHash}(\text{"TapTweak"}, Q \| R)$. G ; a tapscript leaf $L_{\text{spend}} = \langle T \rangle \text{ CSV DROP } \langle pk \rangle \text{ CHECKSIG}$ with a valid Merkle path to R ; a tapscript leaf $L_{\text{data}} = OP_RETURN \langle D \rangle$ with a valid Merkle path to R , where $D = \text{TaggedHash}(\text{"L2/chain_id/v1"}, pk)$ for the L2's own `chain\_id`; and the secret key for pk ; together with a fresh, deterministic nullifier N derived

from the secret key. The mint message m is signed under pk (or by the witness implicitly)."

A proof of this statement reveals V , T , the L2 destination, and the nullifier N ; it conceals o , Q , R , L_{spend} , L_{data} , and pk . The anonymity set is the population of L1 P2TR outputs of value V whose script trees contain compliant L_{spend} and L_{data} leaves with matching chain_id . Fixed denominations of V and fixed allowed values of T are required to keep the anonymity set well-defined; under those constraints, the natural reading of f_{mint} becomes a lookup table over allowed (V, T) tuples rather than a continuous function.

We defer the concrete ZK construction. The point of this subsection is that no element of the L1-side construction — the NUMS internal key, the two-leaf tapscript tree, the tagged-hash binding, the Schnorr signature — needs to change in order to admit the upgrade. The first deployment can be shipped with direct Merkle-path verification on L2 and replaced piecewise with a ZK verifier once a suitable proving system is selected.

7 What This Does Not Address

To be explicit about scope:

- **Consensus.** The L2 needs some way to order transactions. This construction is compatible with BMM, independent PoW, PoS, rollup posting, or any other mechanism the L2 designer prefers.
- **Peg.** BTC does not move into or out of the L2 by this construction. If the L2 wishes to also support pegged BTC, a separate mechanism is required (drivechain, BitVM, federation, etc.), and that mechanism's trust assumptions are independent of this issuance scheme.
- **Token utility.** What L2 tokens *do* — pay fees, govern, represent claims on application state, or merely circulate — is outside the scope of this note.
- **Bootstrap distribution.** The construction issues tokens from genesis onwards via lock-and-mint. There is no premine, no founder allocation, and no snapshot. Whether the L2 designer wishes to add such mechanisms alongside this one is their choice.

8 Open Questions

A few questions deserve more attention than we have given them:

Initial parameter selection. Before the first retarget window completes, r must be set to some value. A poor initial choice causes early retarget windows to swing widely. A conservative bootstrap (longer initial window, tighter adjustment cap) is probably advisable.

Capital concentration. The construction does not directly limit how much a single actor can mint. A well-capitalised participant can in principle lock a large fraction of available BTC and capture a corresponding fraction of L2 issuance, capped at V per UTXO. This is qualitatively the same property exhibited by spacechain (where well-capitalised actors can burn at scale) and by snapshot-forks (where pre-existing BTC distribution determines L2 share). Whether additional limits (per-mint caps, progressively reduced rewards for very large V , etc.) are desirable is a policy question for the L2 designer.

Interaction with fee markets. If the L2 token is also the L2 fee currency, there is a feedback loop between r retargeting and fee dynamics that we have not analysed. The retarget targets mint flow; mint flow affects token supply; token supply affects token price; token price affects fee-paying user incentives. The control-theoretic properties of this loop deserve formal study.

Privacy at scale. The anonymous variant relies on the existence of an anonymity set of unrelated time-locked UTXOs of matching denomination. The size and composition of this set is an empirical question that depends on time-lock usage patterns across the wider Bitcoin economy. Fixed denominations help, but anonymity is ultimately a property of the ambient L1 UTXO distribution, which the L2 cannot control.

9 Conclusion

This note has specified a single component: an issuance primitive. It uses Bitcoin’s existing CSV mechanism to verify time-bounded commitment, applies a sigmoidal minting function justified on first principles (anti-griefing at the short end, monotone-with-sacrifice at the long end), and retargets its single parameter to a chosen issuance pace.

The construction is intentionally narrow, and we believe the narrowness is its strongest feature. By isolating issuance from consensus and from peg, we make the primitive evaluable on its own merits, composable with other proposals, and free of the political baggage that has accumulated around bundled L2 designs over the past decade. The construction requires no Bitcoin soft fork, no miner discretion beyond what Bitcoin already requires, no federation, and no claim that the resulting L2 token is BTC. The user receives a recoverable position rather than an irrevocable sacrifice; the L2 receives a price-discovering, opt-in, fair-launch issuance schedule.

References

- [1] R. Somsen, “Spacechains: Bitcoin sidechains without consensus changes,” Bitcoin technical mailing list and subsequent writeups, 2018–2019.
- [2] P. Wuille, J. Nick, A. Towns, “BIP341: Taproot — SegWit Version 1 Spending Rules,” Bitcoin Improvement Proposal, 2020.
- [3] P. Sztorc, “BIP300: Hashrate Escrows,” Bitcoin Improvement Proposal, 2017.
- [4] P. Sztorc, “BIP301: Blind Merged Mining,” Bitcoin Improvement Proposal, 2019.
- [5] P. Sztorc et al., “eCash: A Bitcoin Hard Fork Activating Drivechains,” ecash.com, 2026.
- [6] J. K. Liu, V. K. Wei, D. S. Wong, “Linkable Spontaneous Anonymous Group Signature for Ad Hoc Groups,” *ACISP*, 2004.
- [7] A. Gibson, “aut-ct: anonymous proof of curve-tree taproot ownership,” open-source software project, 2023–.